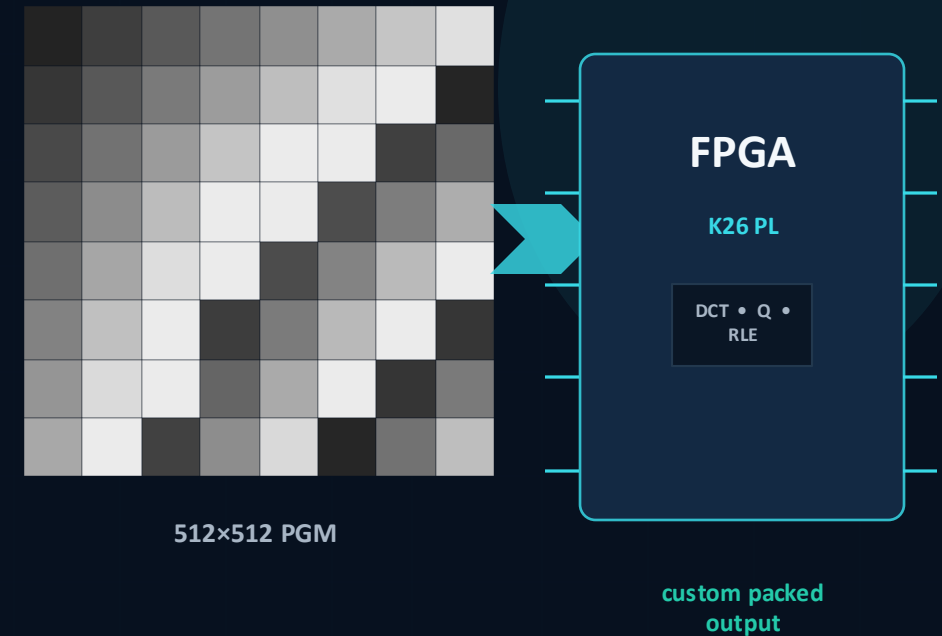


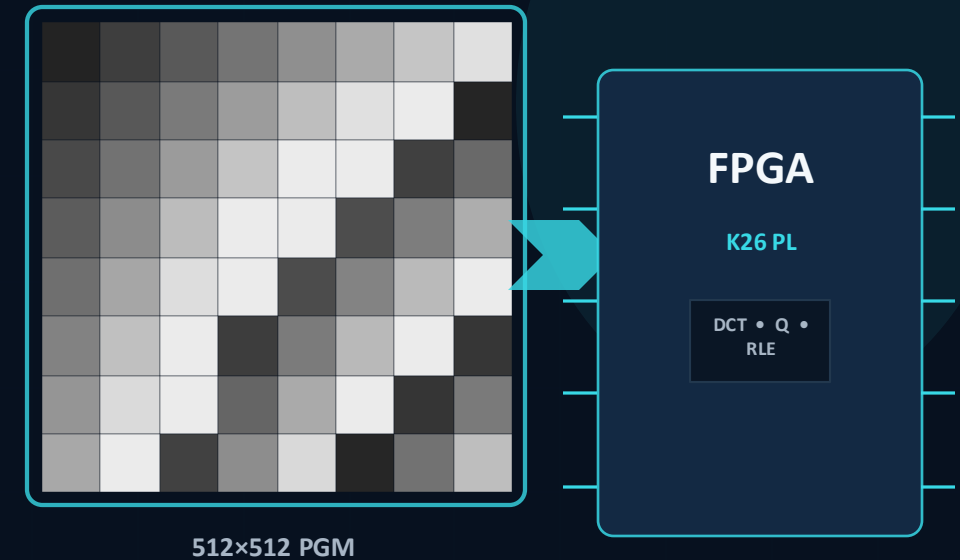
FPGA Acceleration of JPEG-Style Grayscale Image Compression



Abhishek Poddar z5581147
Sakina Huzaifa z5400462
Krishna Mittal z5625451
Shriya Malik z5619441

Can Kria KV260 improve the performance of the complete compression workload?

A complete 8×8 DCT, quantisation, zigzag and packed RLE pipeline — profiled across 41 images on the KV260.



Headline result

DCT and quantisation improved — but serial entropy coding made the current FPGA kernel slower overall.

41

benchmark images
30 natural + 11 synthetic

4.07:1

aggregate FPGA ratio
10.75 MB → 2.64 MB

5.18×

quantisation speedup
largest arithmetic-stage gain

77.9%

kernel time in encoding
the dominant bottleneck

What we set out to build — and how success was judged

The goal was not only to move code into hardware, but to improve the complete compression workload.

Problem

Image compression contains both regular arithmetic and irregular variable-length coding. The first is naturally parallel; the second is much harder to map efficiently to an FPGA.

1

Accelerate the full compressor

Implement the 8×8 processing pipeline in Vitis HLS and run it on the KV260 the KV260 programmable logic.

2

Preserve useful compression

Compare total compressed size and compression ratio across the identical 41-image suite.

3

Measure the whole system

Profile individual stages, transfer overheads and file I/O — not only HLS latency.

Scope

512×5122
8-bit PGM P54,096

8×8 blocks/image

167,936

blocks in suite

Outcome snapshot



Worked

Complete 41-image hardware run; arithmetic stages accelerated; compression ratio retained.



Did not

The current full FPGA implementation did not beat the CPU CPU because packed encoding remained serial.

SUCCESS = SYSTEM-LEVEL SPEEDUP

How the JPEG-style compression pipeline works

The compressor removes visual redundancy first, then represents the remaining information compactly.



Why compression emerges

Lossy step

Quantisation is the intentional quality-compression trade-off. All later stages preserve the quantised coefficients.



Energy concentration

The DCT places most visible structure in a small number of low-frequency coefficients.



Sparsity

Quantisation rounds many high-frequency coefficients to zero, especially in smooth blocks.

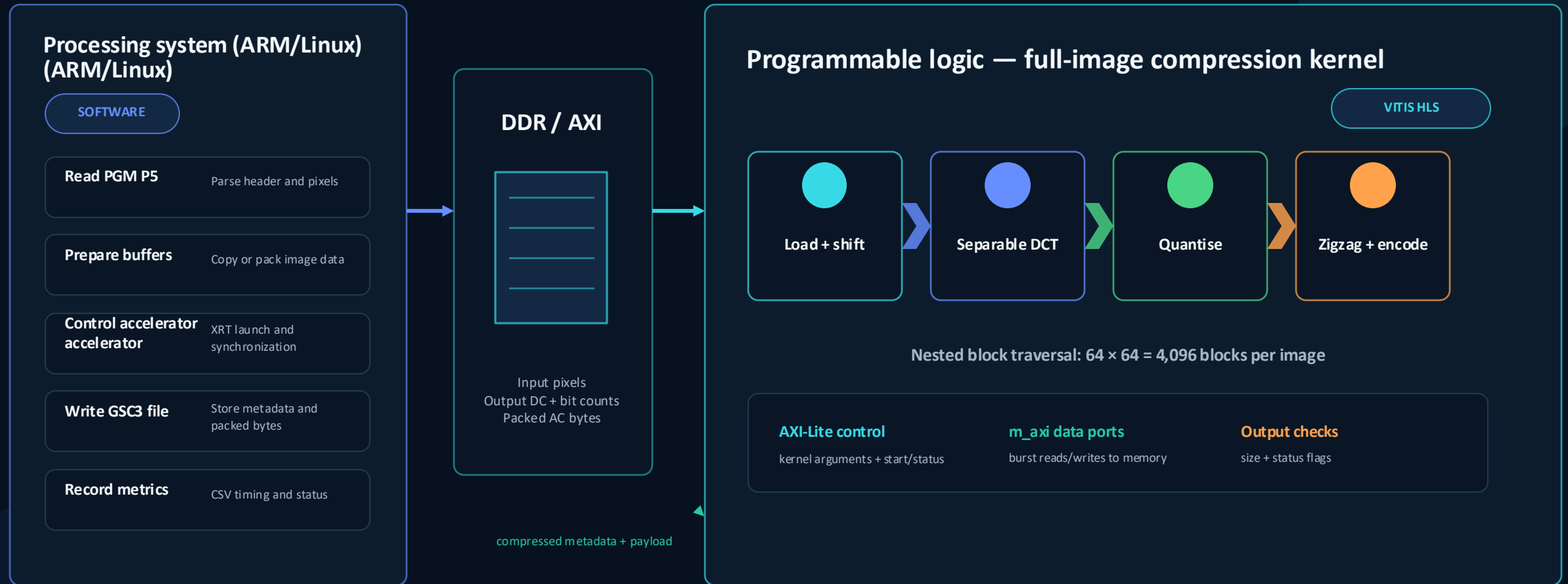


Compact coding

Zigzag scanning groups zeros so RUN/SIZE symbols and EOB markers use fewer bits.

Hardware/software partitioning on the Kria KV260

Linux and XRT manage files and buffers; the programmable logic executes the block-compression kernel.



Where parallelism was introduced — and where it was not

The datapath is highly parallel inside each 8×8 transform, but output generation remains data dependent.

Fixed-point datapath

LOWER OPERATOR COST

Pixels use 9-bit signed values; DCT intermediates use carefully sized fixed-point types; quantised coefficients use 16-bit integers.

Separable 8×8 DCT

128 DOT PRODUCTS

Two one-dimensional matrix passes replace a direct 2D calculation and expose regular multiply-accumulate structure.

Partition + unroll

PARALLEL INNER SUMS

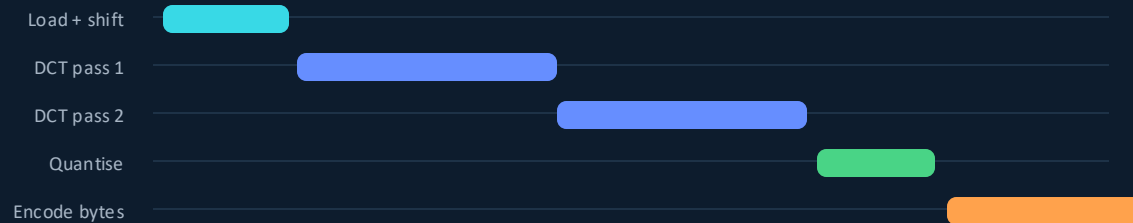
8×8 local arrays are fully partitioned; each eight-term DCT reduction is fully unrolled; output loops are pipelined at II=1.

Packed encoder

VARIABLE LENGTH

Standard zigzag, RUN/SIZE, ZRL and EOB symbols are packed into bytes with per-block status checks.

Conceptual schedule inside one block



Fixed-work arithmetic can be scheduled predictably. Encoder work expands with the number of emitted symbols and bytes.

Remaining serial constraints

TARGET 200 MHz



Block traversal

The current kernel processes blocks through one top-level nested loop.



Bitstream construction

RUN/SIZE emission and packed byte writes depend on coefficient content.

How the CPU and FPGA measurements were collected

The comparison uses identical 512×512 images and per-stage averages rather than a single headline latency.

Benchmark protocol



Dataset

30 natural images plus 11 synthetic edge cases; every file is 512×512 PGM P5.



CPU baseline

One complete software batch with stage-level profiling; 41 images 41 images and 167,936 blocks.



FPGA profiling

One warm-up followed by 10 timed runs per image; average, minimum and maximum recorded.



Metrics

Compressed bytes, compression ratio, load/shift, DCT, quantise, encode, transfers, file writing and status.

ratio = raw bytes / compressed bytes speedup = CPU time / FPGA time

```
Compression complete
Images compressed: 41
Input format: PGM P5
Total raw image size: 10747904 bytes
Total compressed size: 2659049 bytes
Overall compression ratio: 4.042:1
```

Average profiling results

```
Images processed: 41
Total blocks processed: 167936
Image loading: 0.296 ms/image (0.889%)
Level shifting: 1.158 ms/image, 0.283 us/block (3.481%)
2D DCT: 7.760 ms/image, 1.895 us/block (23.331%)
Quantisation: 7.724 ms/image, 1.886 us/block (23.222%)
Zigzag scan: 3.530 ms/image, 0.862 us/block (10.612%)
RLE and bit encoding: 8.872 ms/image, 2.166 us/block (26.675%)
File writing: 2.029 ms/image (6.101%)
Other overhead: 1.892 ms/image (5.688%)
Average total time: 33.260 ms/image
Batch processing time: 1363.658 ms
```

CPU BASELINE EVIDENCE

33.260 ms

ms
CPU total / image

4.042:1

CPU aggregate ratio

Compression effectiveness was essentially preserved

Output size depends strongly on image structure, but the aggregate CPU and FPGA ratios remain very close.

4.042:1

CPU aggregate ratio
2,659,049 compressed bytes

4.072:1

FPGA aggregate ratio
2,639,243 compressed bytes

0.74%

smaller FPGA output
difference is small, not bit-identical

41 / 41

successful status
status = 0 for every image



What the distribution means

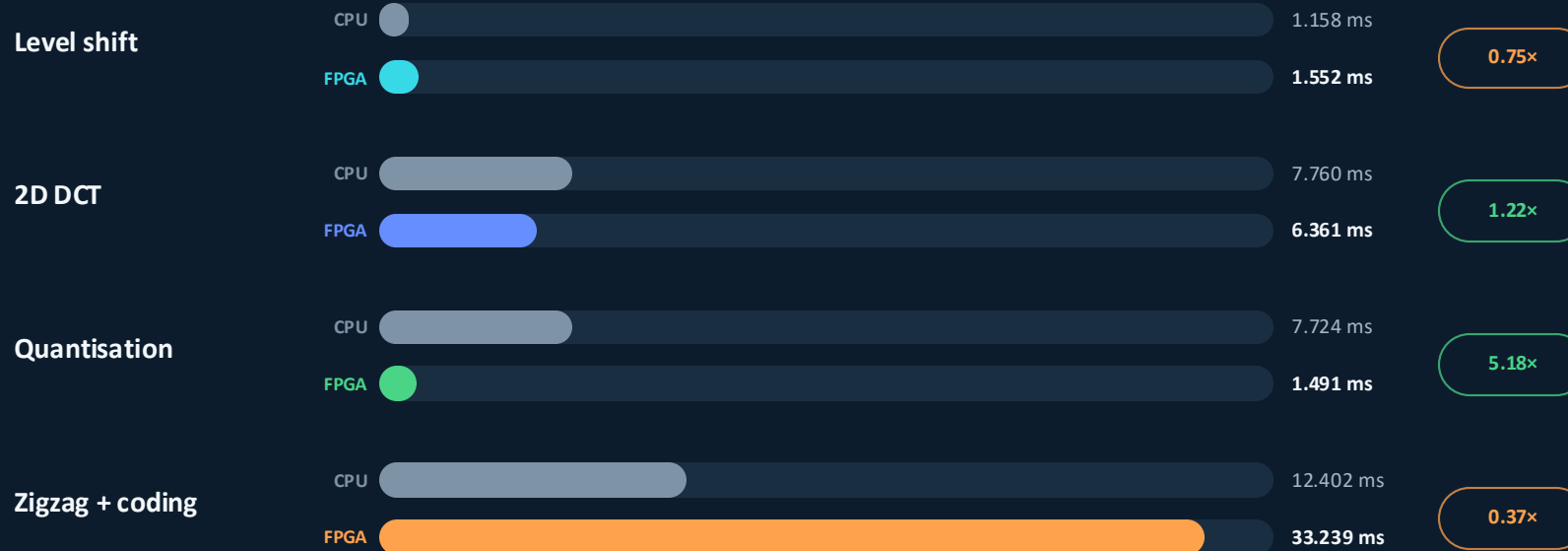
- Highly regular images**
Flat and repetitive patterns reached 12.71:1 because quantisation produced long zero runs.
- Natural images**
The median result was 4.38:1, reflecting mixed smooth areas, edges and texture.
- Random noise**
fixed_seed_noise reached only 1.09:1 because most high-frequency coefficients remained non-zero.

The arithmetic accelerated, but the complete FPGA pipeline did not

CPU and FPGA stage times reveal exactly where the gains were lost.

Average stage time per 512×512 image

Longer bar = more time. Speedup above 1× is faster on FPGA.



CPU coding combines zigzag scan (3.530 ms) and RLE/bit encoding (8.872 ms).

Whole-pipeline view

33.26 ms

CPU complete run

42.65 ms

FPGA kernel only

48.08 ms

estimated FPGA app path

0.78×

kernel vs CPU total

0.69×

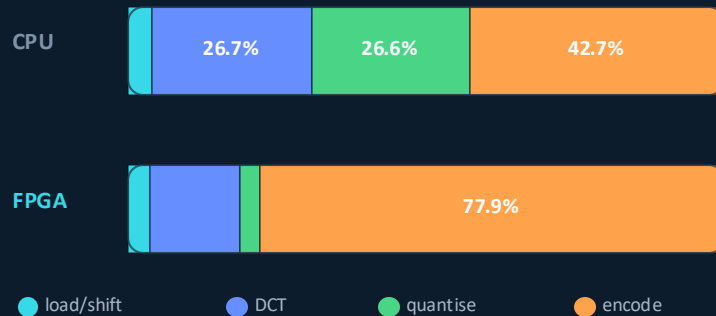
estimated app vs CPU

Why the packed encoder became the system bottleneck

The more bytes an image produces, the longer the FPGA spends emitting its variable-length stream.

Compute-stage composition

Comparing only level shift, DCT, quantisation and coding



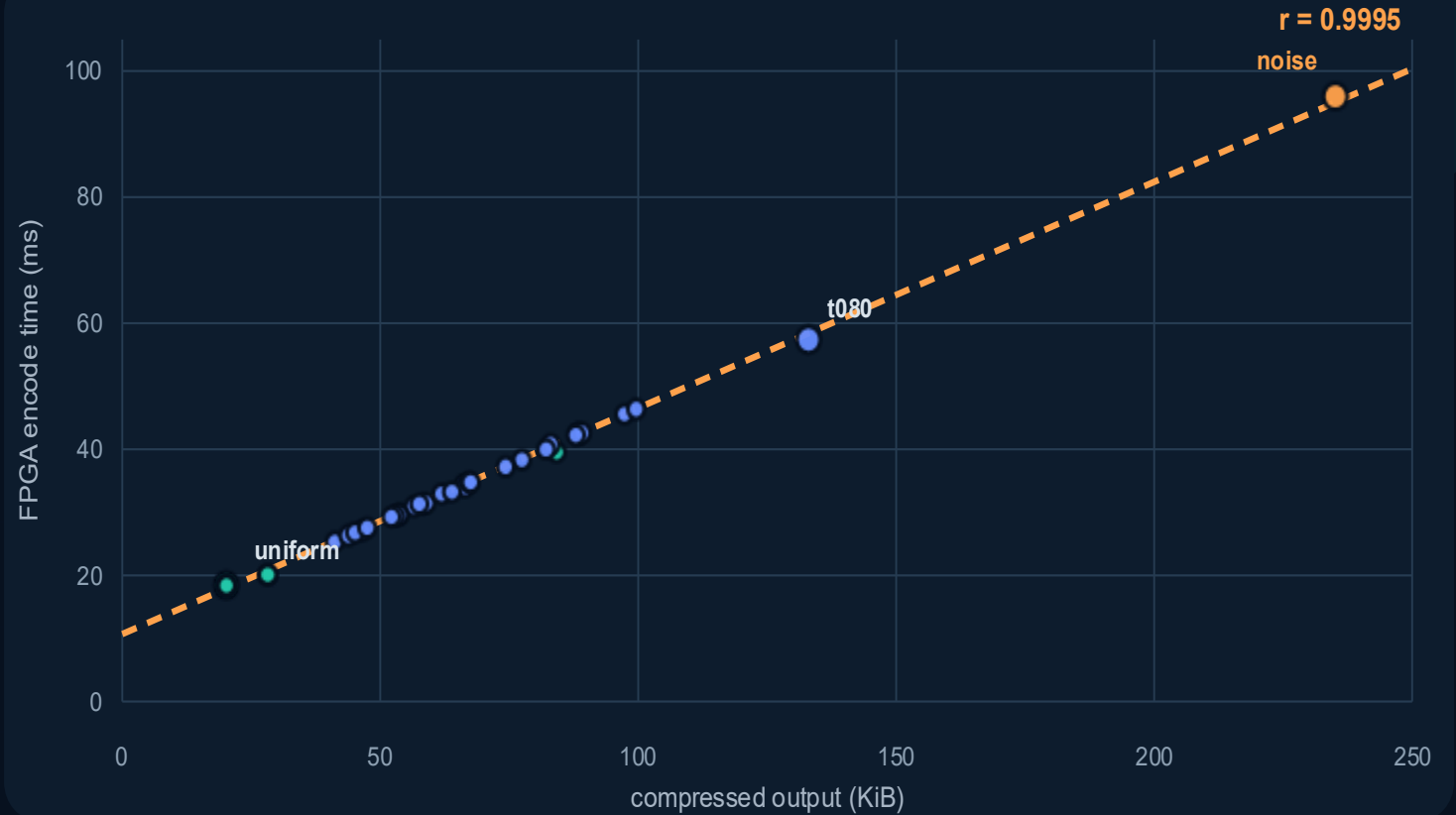
77.9%

FPGA kernel in encode

0.03%

mean min-max spread

The timing is stable. The bottleneck is architectural, not measurement noise.



Compressed size explains encoding time almost perfectly ($r = 0.9995$): more symbols and bytes create more serial packing work.

What worked, what did not — and what we would change next

The project's main lesson is that end-to-end acceleration depends on the slowest stage after optimisation.

What worked



Complete FPGA pipeline

All 41 images completed with successful status and a custom packed output.



Arithmetic acceleration

DCT improved by 1.22× and quantisation by quantisation by 5.18×.



Compression retained

Aggregate ratio remained close: 4.042:1 CPU versus 4.072:1 FPGA.

What did not



No full-system speedup

42.65 ms kernel-only versus 33.26 ms for the CPU complete run.



Serial packed encoding

Variable-length output consumed 77.9% of kernel time.



Limited parallel workload

One block stream and fixed output loops constrained throughput.

Given more time



1 Redesign the writer

Use a byte reservoir and emit whole codewords rather than one bit at a time.



2 Move only valid output

Create a compact contiguous AC stream; avoid fixed-slot byte transfers.



3 Exploit coarse parallelism

Add DATAFLOW, multiple block/CU lanes and batch images to amortise transfers.

Next-iteration roadmap

01

Byte-reservoir coder

02

Compact output stream

03

DATAFLOW between stages

04

Parallel blocks / CUs

05

Post-route power + timing

Final takeaway: accelerating the obvious arithmetic is not enough — the architecture must be designed around the bottleneck that remains.